

Integration Guide

This guide explains how to integrate external open-source UI and multi-agent components into the Chef's Mind AI project. It assumes you have cloned the reference repositories into the `external` directory as described in `docs/EXTERNAL_SOURCES_UI.md`.

Overview

Chef's Mind AI uses several external repositories as references when redesigning its interface and building the multi-agent orchestrator. These sources provide patterns for chat layout, agent navigation, file uploads and multi-agent routing. Only a few directories from each repository are needed. The rest should be ignored.

External repositories

- **lobe-chat** – an open-source, modern design AI Agent Workspace that supports multiple AI providers, knowledge base file uploads (RAG) and a one-click MCP marketplace ¹. Use its layout and component patterns for the new chat UI.
- **open-webui** – a clean chat interface with left and right sidebars, file upload and chat history. Useful for the file uploader and chat input bar.
- **agent-squad** – TypeScript examples for multi-agent routing, including a classifier and supervisor. Use it to upgrade your orchestrator to variant #4.
- **Flowise** – optional. Provides ideas for modular UI and back-end separation. Not required for the new UI.
- **python-prompt-toolkit** – CLI utilities, not used for web UI. Can be repurposed later for an admin CLI.
- **ComfyUI** – a node-based image pipeline. Not used for Chef's Mind UI; may run as a separate service for image workflows.

Integration steps

1. **Prepare the external folder** – ensure your project has an `external` directory containing the cloned repositories listed above. See `docs/EXTERNAL_SOURCES_UI.md` for paths and guidance on which parts to use.
2. **Review the source structure** – open `lobe-chat/src` and `lobe-chat/packages`. Inspect the `layout (app)` and reusable components. Similarly, open `open-webui/src` and `open-webui/backend` to study the file upload and chat patterns.
3. **Build the base layout** – in your `frontend` or `frontend-enhanced` project:
4. Copy the general layout structure from `lobe-chat/src/app/layout.tsx` or similar. Adapt it for Next.js 14 and your existing tailwind configuration.
5. Implement a left sidebar that lists four agents (Accountant, Su-Chef, Research, Media Studio) and a settings button. Use `lobe-chat/src/components` as a reference.
6. Implement a right sidebar with tabs (search, library, calendar) based on `open-webui/src/components`.
7. **Create the chat input bar** – reuse the `open-webui` input bar pattern:
8. Include attachments (paperclip) to upload files.

9. Provide a model selector dropdown (auto, GPT-4o, Gemini, etc.).
10. Include microphone and send buttons.
11. **Integrate multi-agent routing** – use examples from `agent-squad/typescript` to improve your orchestrator:
12. Implement an intent classifier that routes messages to Accountant, Su-Chef, Researcher or Media Studio agents.
13. For complex queries, create a supervisor agent that can call multiple agents in sequence and aggregate results.
14. Adapt the code to your existing `server/agents/orchestrator.ts`.
15. **Add prompt enhancement (optional)** – if you plan to enhance image prompts, create a TypeScript module in `server/media/promptEnhancer.ts` using ideas from `python-prompt-toolkit` or a stable-diffusion prompt reader. Expose it as a tool for the Media Studio agent.
16. **Test the integration** – run your project with `npm run dev` and ensure the new UI loads correctly. Verify that switching agents in the sidebar updates the central chat. Check that file uploads and model selection work as expected.
17. **Remove unused repositories** – after finishing the integration, you can delete `Flowise`, `ComfyUI` and other unused folders from the `external` directory. Keep only those needed for reference.

Additional notes

- The external repositories are references; you are not required to copy their code verbatim. Instead, use them to inspire your own implementations that fit Chef's Mind AI's architecture.
- Keep this file and `docs/EXTERNAL_SOURCES_UI.md` up to date as you migrate components or add new external references.
- Use the `agent-squad` approach (choice #4) for the orchestrator – this keeps the multi-agent system simple and maintainable.

¹ GitHub - lobehub/lobe-chat: LobeHub - an open-source, modern design AI Agent Workspace. Supports multiple AI providers, Knowledge Base (file upload / RAG), one click install MCP Marketplace and Artifacts / Thinking. One-click FREE deployment of your private AI Agent application.
<https://github.com/lobehub/lobe-chat>